Higher Colleges of Technology

كليات التقنية العليا

CDS 2423- PROGRAMMING FOR DATA SCIENCE

Week 7-8

Week 7-8	CLO 3 Create scripts to solve business problems using data analysis techniques such as classification, clustering, optimization, and recommendation

Document Revision CONTROL (DRC)

Version	Author/Reviewer	Effective Date	Change Description	DRC No
1.0	Mohammed Abdul Rahim	2/05/2023	Define the first version	001
1.0	Amala Rajan	28/10/2023	Reviiew 1st version	001

X.Y
X: Major change
Y: minor change

Lecture Outline

- Regression
- Simple Linear Regression
- Multiple Linear Regression
- Polynomial Regression
- Regression - Model Selection
- Python Programming for Regression

LECTURE OBJECTIVES

- At the conclusion of the lecture, the student should be able to perform the tasks below:
  - Comprehension of the concept of regression
  - Able to investigate the different forms of regression
  - Understand choosing the best model
  - Learn to write Python code for various types of regression equations

Regression Analysis

- **Regression analysis** is a statistical method that shows the relationship between two or more variables.
  - **Correlation analysis** determines if a relationship exists between variables, whereas **regression analysis** predicts one variable based on the other.
- **Types of Regression:**
- **Bivariate Regression:** Analysis of two variables to determine relationships (e.g., Simple Linear Regression).
- **Multivariate Regression:** Analysis of many variables to determine relationships (e.g., Multiple Linear Regression).

Linear Regression

- **Linear regression analysis** is used to predict the value of a variable based on the value of another variable.
  - The variable you want to predict is called the **dependent variable**.
  - The variable you are using to predict the other variable's value is called the **independent variable**.

Simple Linear Regression – Equation

- The simplest form of a regression equation is the linear regression equation, which is expressed as:

$$\underbrace{\hat{y}}_{\text{Dependent variable}} = \underbrace{b_0}_{\text{y-intercept (constant)}} + \underbrace{b_1}_{\text{Slope coefficient}} \underbrace{X_1}_{\text{Independent variable}}$$

- $y = a + bX$

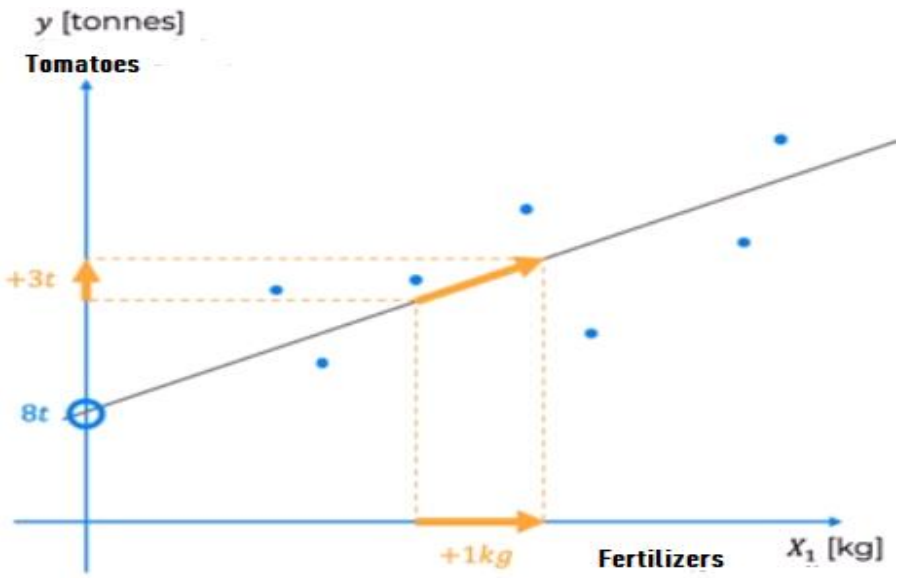
Simple Linear Regression – Bi Variate

- The yield of tomatoes depends on the amount of fertilizer. Here,
  - the yield of tomatoes is dependent variable and
  - the amount of fertiliser is an independent variable.
- The yield of tomatoes for different amounts of fertilizer in future growing seasons using a regression equation.

Y = a + b X

Independent(X) → Dependent(Y)

a= 8
b=3



Tomatoes = $8 + 3 \times \text{Fertilizers}$

If there are no fertilizers, 8 kg of tomatoes will be obtained.

For 1 kg of fertilizer, 3 kg of tomatoes are obtained.

Simple Linear Regression– Dataset

- **Dataset Used:** Power_Plant.CSV
- Predicts dependent variables based on the number of independent variables.
  - Uses simple linear or polynomial regression for one independent variable.
  - Uses multiple linear regression for multiple variables.

Temperature	Vaccum	Pressure	Relative_humidity	Energy
14.96	41.76	1024.07	73.17	463.26
25.18	62.96	1020.04	59.08	444.37
5.11	39.4	1012.16	92.14	488.56
20.86	57.32	1010.24	76.64	446.48
10.82	37.5	1009.23	96.62	473.9
26.27	59.44	1012.23	58.77	443.67
15.89	43.96	1014.02	75.24	467.35
9.48	44.71	1019.12	66.43	478.42
14.64	45	1021.78	41.25	475.98

Independent variables

Dependent variables

Simple Linear Regression – Python

Steps

1. Importing the libraries
2. Importing the dataset
3. Splitting the dataset into the Training set and Test set
4. Training the Simple Linear Regression model on the Training set
5. Predicting the Test set results
6. Predicting the Specific Value
7. Visualising the Training set results
8. Visualising the Test set results
9. Getting the Final Linear Regression equation

Simple Linear Regression – Python

Outline

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Load the data
data = pd.read_csv('data.csv')

# Separate the independent and dependent variables
X = data['independent_variable']
y = data['dependent_variable']

# Create and fit the linear regression model
model = LinearRegression()
model.fit(X, y)

# Make predictions
predictions = model.predict(X)

# Plot the data and the regression line
plt.scatter(X, y)
plt.plot(X, predictions)
plt.show()
```

Simple Linear Regression – Python

Importing the libraries

```
[ ] import numpy as np
    import matplotlib.pyplot as plt
    import pandas as pd
```

Simple Linear Regression – Python

Importing the dataset

Pressure column

Independent variable

Dependent variable

```
[14] dataset = pd.read_csv('Power_Plant.csv')
     X = dataset.iloc[:, [2]].values
     y = dataset.iloc[:, -1].values
     print(X)
     print(y)
```

Last column(Energy)

Files

..

sample_data

Power_Plant.csv

```
[[1024.07]
 [1020.04]
 [1012.16]
 ...
 [1012.92]
 [1013.86]
 [1017.23]]
[463.26 444.37 488.56 ... 429.57 435.74 453.28]
```

Simple Linear Regression – Python

Splitting the dataset into the Training set and Test set

```
[15] from sklearn.model_selection import train_test_split
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 1/3, random_state = 0)
```

Test size is 1/3(33%). So, training size is 2/3.

Simple Linear Regression – Python

Training the Simple Linear Regression model on the Training set

```
[ ] from sklearn.linear_model import LinearRegression
    regressor = LinearRegression()
    regressor.fit(X_train, y_train)
```

Configuring linear regression

Applying the regression configuration to training set

Simple Linear Regression – Python

Predicting the Test set results

```
y_pred = regressor.predict(X_test)
print(X_test)
print(y_pred)
```

```
[[1009.56]
 [1021.51]
 [1019.21]
 ...
 [1013.39]
 [1007.13]
 [1007.38]]
[448.90397264 466.84299321 463.39029469 ... 454.65346626 445.2561216
 445.63141491]
```

Simple Linear Regression – Python

Predicting the Specific Value

```
[18] # find the energy(dependent variable) for pressure(ind var)=1024.07
     regressor.predict([[1024.07]])
```

```
array([470.68599678])
```

Temperature	Vaccum	Pressure	Relative_humidity	Energy
14.96	41.76	1024.07	73.17	463.26
25.18	62.96	1020.04	59.08	444.37
5.11	39.4	1012.16	92.14	488.56
20.86	57.32	1010.24	76.64	446.48
10.82	37.5	1009.23	96.62	473.9
26.27	59.44	1012.23	58.77	443.67
15.89	43.96	1014.02	75.24	467.35
9.48	44.71	1019.12	66.43	478.42
14.64	45	1021.78	41.25	475.98

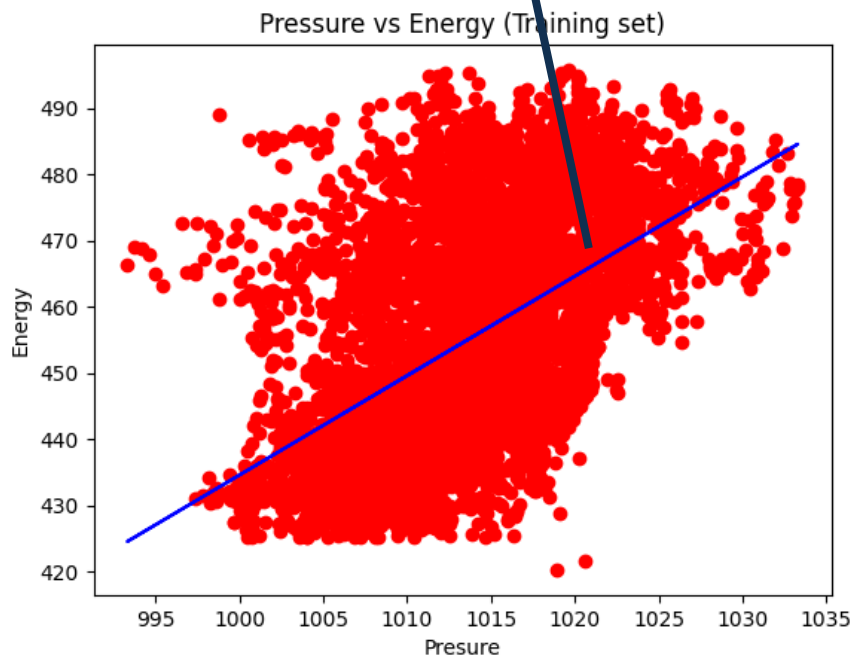
The predicted value(470) is close to actual value(463). To improve the closeness, the model should be optimized

Simple Linear Regression – Python

Visualising the Training set results

```
[19] plt.scatter(X_train, y_train, color = 'red')
     plt.plot(X_train, regressor.predict(X_train), color = 'blue')
     plt.title('Pressure vs Energy (Training set)')
     plt.xlabel('Presure')
     plt.ylabel('Energy')
     plt.show()
```

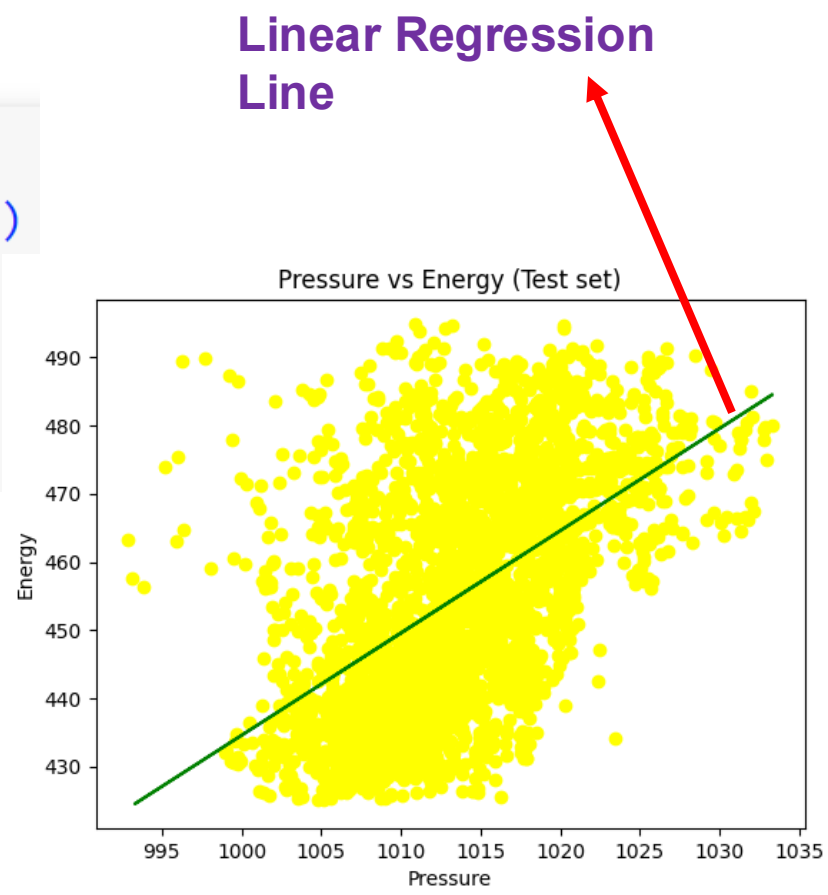
Linear
Regression Line



Simple Linear Regression – Python

Visualising the Test set results

```
plt.scatter(X_test, y_test, color = 'yellow')
plt.plot(X_train, regressor.predict(X_train), color = 'green')
plt.title('Pressure vs Energy (Test set)')
plt.xlabel('Pressure')
plt.ylabel('Energy')
plt.show()
```



Simple Linear Regression – Python

Getting the Final Linear Regression equation

```
[7] print(regressor.coef_)
    print(regressor.intercept_)
```

```
[1.50117327]
-1066.6205133949695
```

$$\hat{y} = b_0 + b_1 X_1$$

Dependent variable

y-intercept (constant)

Slope coefficient

Independent variable

Linear Regression equation is

Energy = –1066.62051 + 1.5011 * Pressure

Simple Linear Regression – Python

Accuracy of the Linear Regression equation using R-squared

```
from sklearn.metrics import r2_score
print(r2_score(y_test, y_pred))
```

```
0.2611671708679816
```

Only 26% accuracy to predict the value using this regression model

Simple Linear Regression – Python

Energy = –1066.62051 + 1.5011 * Pressure

Based on the Linear Regression equation calculate energy for the below pressure:

Pressure	Energy
1010.74	
1014.02	
1015.14	

Simple Linear Regression – Python

Energy = –1066.62051 + 1.5011 * Pressure

Based on the Linear Regression equation calculate energy for the below pressure:

Pressure	Energy
1010.74	450.601304
1014.02	455.524912
1015.14	457.206144

Multiple Linear Regression – Multi variate

- Multiple Linear regression analysis is used to predict the values of a dependent variable based on the values of more than one independent variable.
- Multiple linear regression equation is :

$$Y = a + b_1 X_1 + b_2 X_2 \ldots + b_n X_n$$

where Y is dependent and $X_1, X_2, \ldots X_n$ were independent variables

Multiple Linear Regression – Multi variate

$$\hat{y} = b_0 + b_1 X_1 + b_2 X_2 + \cdots + b_n X_n$$

Dependent variable

y-intercept (constant)

Slope coefficient 1

Independent variable 1

Slope coefficient 2

Independent variable 2

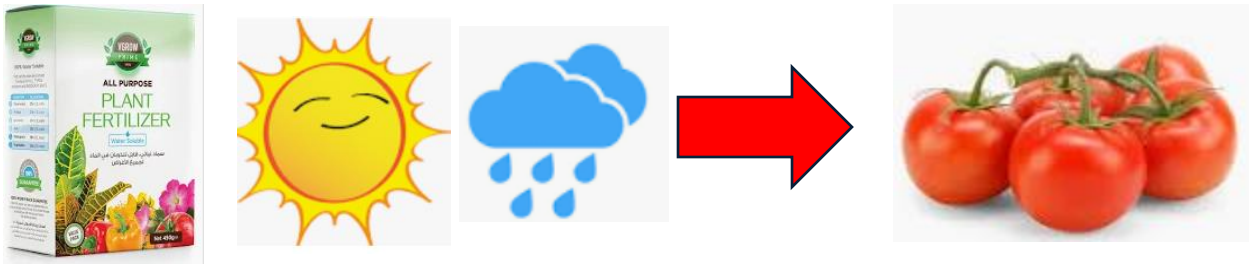
Slope coefficient n

Independent variable n

Multiple Linear Regression – Multi variate

- The yield of tomatoes depends on the amount of fertilizer, temperature and amount of rain. Here,
  - the yield of tomatoes is dependent variable and
  - the amount of fertilizer, temperature and rain are independent variables.
- The yield of tomatoes for different amounts of fertiliser in future growing seasons using a regression equation.

Independent(X) Dependent(Y)



$$Y = a + b_1 X_1 + b_2 X_2 \ldots + b_n X_n$$

$$\text{Tomatoes} = a + b_1 * \text{Fertilizers} + b_2 * \text{Temp} + b_3 * \text{Rain}$$

$$\text{Tomatoes} = 8 + 3 * \text{Fertilizers} - 0.25 * \text{Temp} + 1.05 * \text{Rain}$$

https://www.mdpi.com/2073-4395/11/5/885

Multiple Linear Regression – Python

Steps

1. Importing the libraries
2. Importing the dataset
3. Splitting the dataset into the Training set and Test set
4. Training the Multiple Linear Regression model on the Training set
5. Predicting the Test set results
6. Predicting the Specific Value
7. Getting the Final Multiple Linear Regression equation

Multiple Linear Regression – Python

Outline

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Load the data
data = pd.read_csv('data.csv')

# Separate the independent and dependent variables
X = data[['independent_variable1', 'independent_variable2']]
y = data['dependent_variable']

# Create and fit the linear regression model
model = LinearRegression()
model.fit(X, y)

# Make predictions
predictions = model.predict(X)
```

Multiple Linear Regression – Python

Importing the libraries

```
[ ] import numpy as np
    import matplotlib.pyplot as plt
    import pandas as pd
```

Multiple Linear Regression – Python

Importing the dataset

All columns except last column

Independent variables

```
[ ] dataset = pd.read_csv('Power_Plant.csv')
    X = dataset.iloc[:, :-1].values
    y = dataset.iloc[:, -1].values
```

Dependent variables

Last column(Energy)

```
[[1024.07]
 [1020.04]
 [1012.16]
 ...
 [1012.92]
 [1013.86]
 [1017.23]]
[463.26 444.37 488.56 ... 429.57 435.74 453.28]
```

X

y

Temperature	Vaccum	Pressure	Relative_humidity	Energy
14.96	41.76	1024.07	73.17	463.26
25.18	62.96	1020.04	59.08	444.37
5.11	39.4	1012.16	92.14	488.56
20.86	57.32	1010.24	76.64	446.48
10.82	37.5	1009.23	96.62	473.9
26.27	59.44	1012.23	58.77	443.67
15.89	43.96	1014.02	75.24	467.35
9.48	44.71	1019.12	66.43	478.42
14.64	45	1021.78	41.25	475.98

Independent variables

Dependent variables

Multiple Linear Regression – Python

Splitting the dataset into the Training set and Test set

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

Multiple Linear Regression – Python

Training the Multiple Linear Regression model on the Training set

```
[ ] from sklearn.linear_model import LinearRegression
    regressor = LinearRegression()
    regressor.fit(X_train, y_train)
```

Configuration for Multiple linear regression

Applying multiple linear regression to train data set

Multiple Linear Regression – Python

Predicting the Test set results

```
[ ] y_pred = regressor.predict(X_test)
    np.set_printoptions(precision=2)
    print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))
```

Multiple Linear Regression – Python

Predicting the Specific Value

```
[ ] # find the energy(dependent variable) for temp,vaccum,pressure and humidity(ind vars)
    regressor.predict([[14.96,41.76,1024.07,73.1]])
```

```
array([467.31])
```

Temperature	Vaccum	Pressure	Relative humidity	Energy
14.96	41.76	1024.07	73.17	463.26
25.18	62.96	1020.04	59.08	444.37
5.11	39.4	1012.16	92.14	488.56
20.86	57.32	1010.24	76.64	446.48
10.82	37.5	1009.23	96.62	473.9
26.27	59.44	1012.23	58.77	443.67
15.89	43.96	1014.02	75.24	467.35
9.48	44.71	1019.12	66.43	478.42
14.64	45	1021.78	41.25	475.98

Independent variables

Dependent variables

Actual value = 463.26
The model predicted as 467.31

Multiple Linear Regression – Python

Getting the Final Multi Linear Regression Equation

```
print(regressor.coef_)
print(regressor.intercept_)
```

```
[-1.97 -0.24  0.06 -0.16]
452.84103716163844
```

Thus, the multiple linear Equation is:

Energy = 452.84103716163844 -1.97 * temperature -0.24* vacuum 0.06*pressure -0.16*humidity

Multiple Linear Regression – Python

Based on the multiple linear Regression equation calculate energy for the below temperature, vacuum, pressure and humidity:

Energy = 452.841037161638 44 -1.97 * temperature -0.24* vacuum 0.06*pressure -0.16*humidity

Temperature	Vaccum	Pressure	Relative_humidity	Energy
14.96	41.76	1024.07	73.17	
25.18	62.96	1020.04	59.08	
5.11	39.4	1012.16	92.14	

Multiple Linear Regression – Python

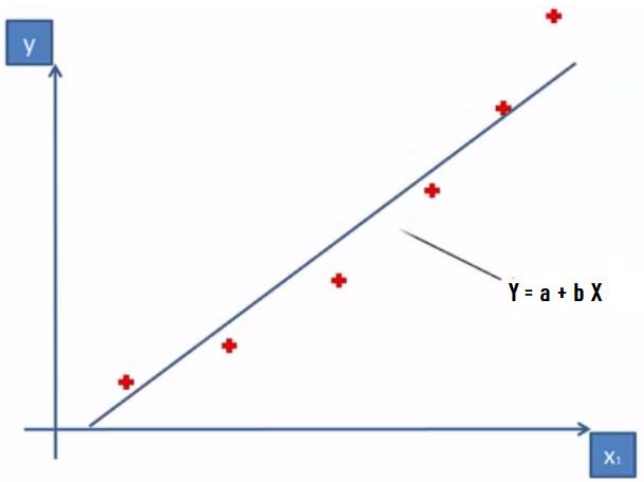
Based on the multiple linear Regression equation calculate energy for the below temperature, vacuum, pressure and humidity:

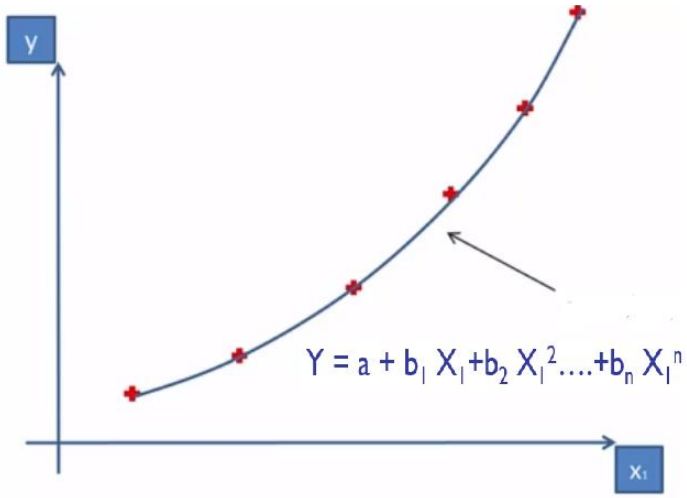
Energy = 452.841037161638 44 -1.97 * temperature -0.24* vacuum 0.06*pressure -0.16*humidity

Temperature	Vaccum	Pressure	Relative_humidity	Energy
14.96	41.76	1024.07	73.17	
25.18	62.96	1020.04	59.08	
5.11	39.4	1012.16	92.14	

Polynomial Linear Regression – Non Linear

- Similar like linear regression with one dependent and one independent variable
- Sometimes in linear regression equation straight line will not fit since some data points much deviate from the straight line
- In such a case, polynomial linear regression is used to make parabolic path instead of straight line to fit all the data points
- Polynomial linear regression equation is:
- $Y = a + b_1 X_1 + b_2 X_1^2 \ldots + b_n X_1^n$
- where Y is dependent and X_1 is independent variable





Polynomial Linear Regression – Python

Polynomial Regression

- Importing the libraries
- Importing the dataset
- Splitting the dataset into the Training set and Test set
- Training the Polynomial Regression model on the Training set
- Predicting the Test set results
- Predicting the Specific Value
- Evaluating the Model Performance
- Getting the Final Polynomial Regression Equation

Polynomial Linear Regression – Python

Importing the libraries

```
[2] import numpy as np
    import matplotlib.pyplot as plt
    import pandas as pd
```

Polynomial Linear Regression – Python

Importing the dataset

Pressure column

Independent variable

Dependent variable

```
[14] dataset = pd.read_csv('Power_Plant.csv')
     X = dataset.iloc[:, [2]].values
     y = dataset.iloc[:, -1].values
     print(X)
     print(y)
```

Last column(Energy)

```
[[1024.07]
 [1020.04]
 [1012.16]
 ...
 [1012.92]
 [1013.86]
 [1017.23]]
[463.26 444.37 488.56 ... 429.57 435.74 453.28]
```

Polynomial Linear Regression – Python

Splitting the dataset into the Training set and Test set

```
[ ] from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

Test set is 20% and hence training is 80%

Polynomial Linear Regression – Python

Training the Polynomial Regression model on the Training set

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
poly_reg = PolynomialFeatures(degree = 4)
X_poly = poly_reg.fit_transform(X_train)
regressor = LinearRegression()
regressor.fit(X_poly, y_train)
```

Polynomial configuration

Degree of polynomial is 4

$Y = a + b_1 X_1 + b_2 X_1^2 \ldots + b_n X_1^4$

Applying polynomial configuration to train data set

Polynomial Linear Regression – Python

Predicting the Test set results

```
[ ] y_pred = regressor.predict(poly_reg.transform(X_test))
    np.set_printoptions(precision=2)
    print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))
```

```
[[446.7  431.23]
 [469.43 460.01]
 [465.26 461.14]
 ...
 [472.67 473.26]
 [442.52 438.  ]
 [443.77 463.28]]
```

Reshape function coverts data into array format without changing the data

Polynomial Linear Regression – Python

Predicting the Specific Value

```
# find the energy(dependent variable) for pressure(ind var)=1024.07
regressor.predict(poly_reg.fit_transform([[1024.07]]))
```

```
array([473.06])
```

Temperature	Vaccum	Pressure	Relative_humidity	Energy
14.96	41.76	1024.07	73.17	463.26
25.18	62.96	1020.04	59.08	444.37
5.11	39.4	1012.16	92.14	488.56
20.86	57.32	1010.24	76.64	446.48
10.82	37.5	1009.23	96.62	473.9
26.27	59.44	1012.23	58.77	443.67
15.89	43.96	1014.02	75.24	467.35
9.48	44.71	1019.12	66.43	478.42
14.64	45	1021.78	41.25	475.98

The predicted value(470) is close to actual value(463). To improve the closeness, the model should be optimize

Polynomial Linear Regression – Python

Evaluating the Model Performance

```
[ ] from sklearn.metrics import r2_score
    r2_score(y_test, y_pred)
```

```
0.2960206068764252
```

R^2 value is useful to choose the best model for the given data set

Polynomial Linear Regression – Python

Getting the Final Polynomial Regression Equation

```
[8] print(regressor.coef_)
    print(regressor.intercept_)
```

```
[ 0.00e+00 -1.11e-02 -7.50e+00  9.84e-03 -3.63e-06]
1289269.0209504422
```

Thus, the polynomial linear regression equation is:

Energy = 1289269.0209504422-1.11e-02*pressure^1 -7.50e+00* pressure^2 +9.84e-03*pressure^3 -3.63e-06*pressure^4

Regression – Model Selection

- For the given dataset, the following regression models are designed:
  - Multi linear regression model
  - Poly linear regression model
- Then, how can you choose which model will be best fit for the given dataset
- **R^2 is used to chose the best fit model for regression**

Regression – Model Selection

- **R-squared** shows how well the data fit the regression model (the goodness of fit).
- R-squared can take any values between **0 to 1**
- A high r-squared indicates a good fit, but an extremely high fit can raise questions (perfectly linear model !).
- A low r-squared figure is generally a bad sign for predictive models. However, in some cases, the best linear model that can be found may still show a small value.

Regression – Best Fit

Evaluating the Model Performance

```
[ ] from sklearn.metrics import r2_score
    r2_score(y_test, y_pred)
```

```
0.2960206068764252
```

R^2 value for polynomial regression is 29%

Evaluating the Model Performance

```
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

```
0.9325315554761303
```

R^2 value for multiple linear regression is 93%

So, the best fit regression model for Power_Plant dataset is Multiple Linear Regression Model

Project

13.	Build, Train, Develop and Evaluate using Simple Regression for chosen dataset.
14.	Develop a script to forecast the value of the dependent variable from all the relevant independent variables using Multiple Linear Regression

Skill Yourself – Other Regression Models

Regression Model	Description	Evaluation Methods
Linear Regression	Models the relationship between a dependent variable and one or more independent variables using a linear 1 equation.	Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R-squared, Adjusted R-squared, Mean Absolute Error (MAE)
Logistic Regression	Used for binary classification problems, predicting the probability of a binary **outcome**.	Accuracy, Precision, Recall, F1-score, AUC-ROC, Log Loss
Polynomial Regression	Extends linear regression by fitting a polynomial equation to the data, capturing more complex relationships. 2	Same as Linear Regression
Ridge Regression	Addresses multicollinearity by adding a penalty term to the regression equation.	Same as Linear Regression
Lasso Regression	Similar to Ridge Regression, but uses a different penalty term that can also perform feature selection.	Same as Linear Regression
Elastic Net Regression	Combines the penalties of Ridge and Lasso Regression.	Same as Linear Regression
Support Vector Regression (SVR)	Uses support vectors to find the best fit line that maximizes the margin between the line and the data points.	Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R-squared
Decision Tree Regression	Creates a tree-like model of decisions and their possible consequences to predict the value of a continuous target variable.	Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R-squared
Random Forest Regression	An ensemble learning method that combines multiple decision trees to improve prediction accuracy.	Same as Decision Tree Regression
Gradient Boosting Regression	Another ensemble learning method that builds an additive model in a forward stage-wise fashion.	Same as Decision Tree Regression

Covered in this course

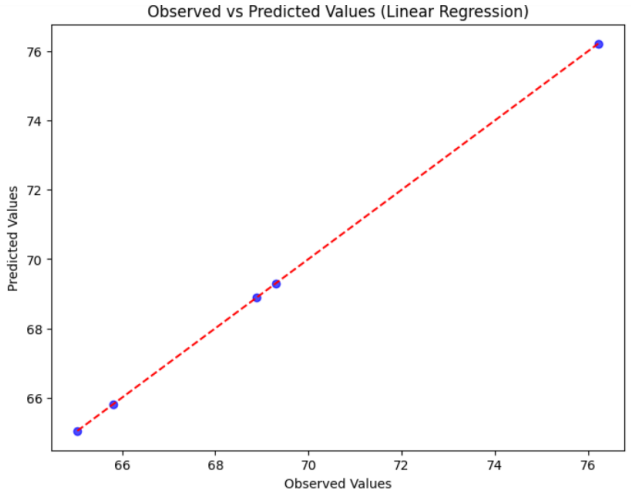
Do It Yourself - I

- Develop and compare regression models to
  - Predict the 'Total' marks based on 'Quiz', 'Practical', and 'Project' scores.
  - Implement Multiple Linear Regression, Polynomial Regression, Lasso, Ridge, and Elastic Net models.
  - Select the best-fitting model based on evaluation metrics.
  - Analyze the model's performance by comparing predicted 'Total' marks with the actual 'Total' marks in the dataset.

```
Comparison of Regression Models:
                            Model  Mean Squared Error  R-squared
0                Linear Regression        0.000000e+00   1.000000
1  Polynomial Regression (Degree 2)        1.684250e-26   1.000000
2                 Ridge Regression        1.750587e-05   0.999999
3                 Lasso Regression        4.692068e-04   0.999970
4            Elastic Net Regression        2.068225e-04   0.999987

Best-Fit Model: Linear Regression
R-squared: 1.0
Mean Squared Error: 0.0

Regression Equation for Linear Regression:
y = 0.00 + (0.10)*Quiz + (0.30)*Practical + (0.30)*Project + (0.30)*Final
```



Do It Yourself - 2

Try other regression model using ChatGPT for the power plant dataset and choose the best.

Power_Plant.csv
Spreadsheet

Write Python code to create a regression model like multiple linear, polynomial, ridge, lasso, elastic net, and gradient boosting for the attached dataset. The program should include printing the r2_score, regression intercept, and coefficient for each model. It should also include the code to predict the values for each model and print the predicted value.

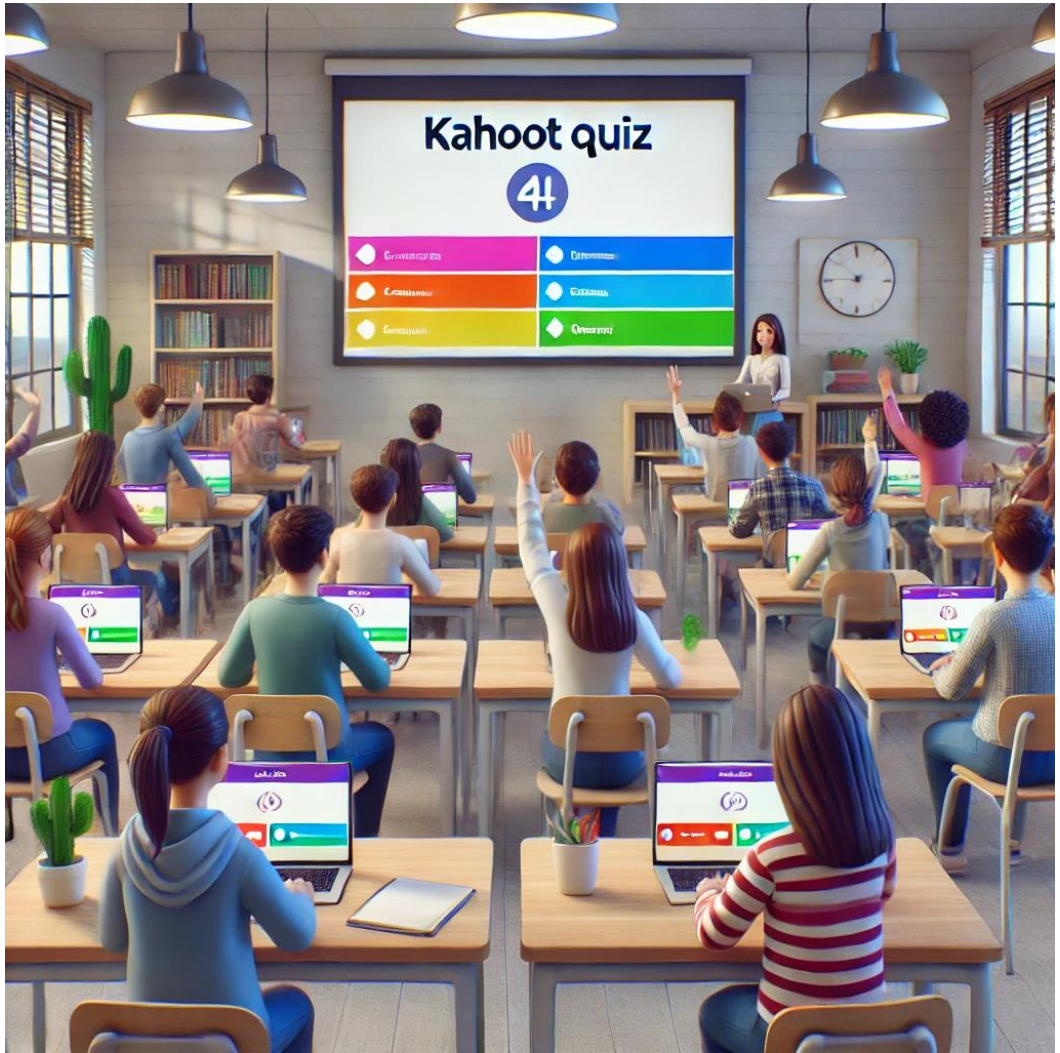
After executing the above,

provide the Python code

Conclusion

- Regression is a key statistical tool for data analysis to predict or forecast the value.
- **Simple Linear Regression** uses one independent variable.
- **Polynomial Regression** uses one independent variable but much improvement than simple linear regression.
- **Multiple Linear Regression** uses multiple independent variables.
- Python can be used to implement and visualize regression models effectively.

Kahoot Time!



Higher Colleges of Technology

كليات التقنية العليا

Thank You